

CSA0604-DESIGN AND ANALYSIS OF ALGORITHMS

NAME: SANTHIYA S

REG NO: 192511360

1. def firstPalindrome(words):

for word in words:

if word == word[::-1]:

return word

return ""

Example 1

words = ["abc", "car", "ada", "racecar", "cool"]

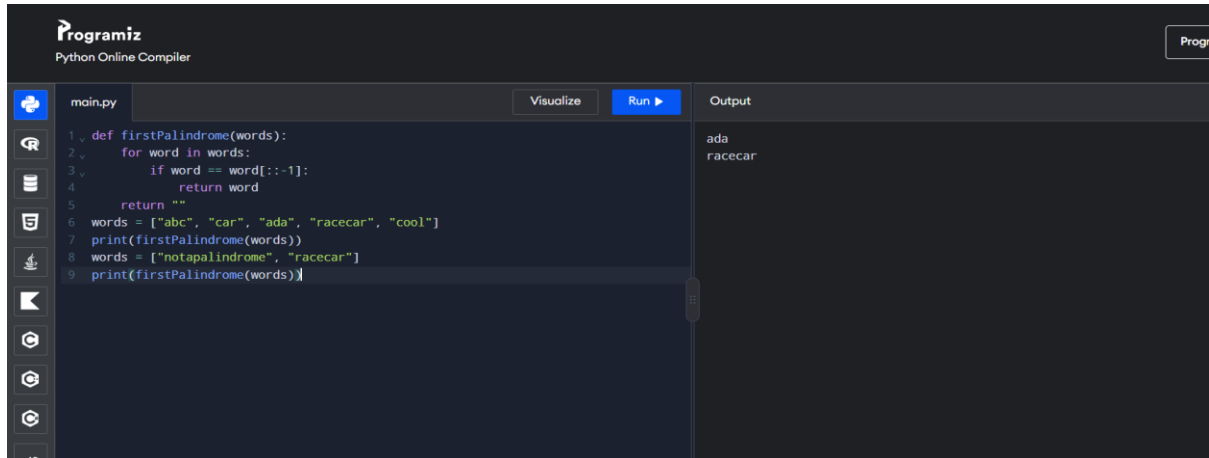
print(firstPalindrome(words))

Example 2

words = ["notapalindrome", "racecar"]

print(firstPalindrome(words))

OUTPUT:



```
Programiz
Python Online Compiler

main.py Visualize Run Output
1. def firstPalindrome(words):
2.     for word in words:
3.         if word == word[::-1]:
4.             return word
5.     return ""
6. words = ["abc", "car", "ada", "racecar", "cool"]
7. print(firstPalindrome(words))
8. words = ["notapalindrome", "racecar"]
9. print(firstPalindrome(words))

ada
racecar
```

2. def findIntersectionValues (nums1, nums2):

answer1 = 0

answer2 = 0

for i in nums1:

if i in nums2:

answer1 += 1

```

    for i in nums2:
        if i in nums1:
            answer2 += 1

    return [answer1, answer2]

# Example 1
nums1 = [2, 3, 2]
nums2 = [1, 2]
print(findIntersectionValues(nums1, nums2))

# Example 2
nums1 = [4, 3, 2, 3, 1]
nums2 = [2, 2, 5, 2, 3, 6]
print(findIntersectionValues (nums1, nums2))

```

OUTPUT:

The screenshot shows the Programiz Python Online Compiler interface. The code editor on the left contains the following Python code:

```

1 def findIntersectionValues(nums1, nums2):
2     answer1 = 0
3     answer2 = 0
4     for i in nums1:
5         if i in nums2:
6             answer1 += 1
7     for i in nums2:
8         if i in nums1:
9             answer2 += 1
10    return [answer1, answer2]
11 nums1 = [2, 3, 2]
12 nums2 = [1, 2]
13 print(findIntersectionValues(nums1, nums2))
14 nums1 = [4, 3, 2, 3, 1]
15 nums2 = [2, 2, 5, 2, 3, 6]
16 print(findIntersectionValues(nums1, nums2))

```

The 'Run' button is highlighted in blue. The output panel on the right displays the results of the execution:

```

[2, 1]
[3, 4]

```

3. def sumCounts(nums):

```

total = 0

for i in range(len(nums)):
    distinct = set()

    for j in range(i, len(nums)):
        distinct.add(nums[j])

    count = len(distinct)

    total += count * count

return total

```

Example 1

```
nums = [1, 2, 1]
```

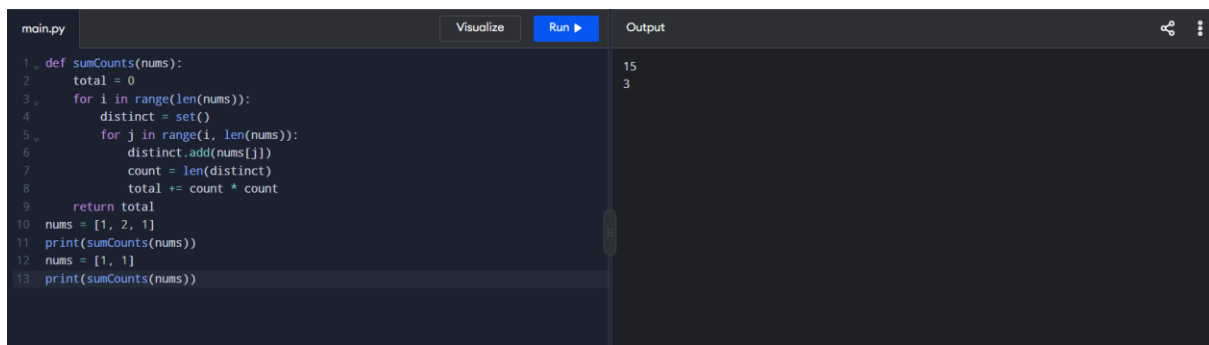
```
print(sumCounts(nums))
```

Example 2

```
nums = [1, 1]
```

```
print(sumCounts(nums))
```

OUTPUT:

A screenshot of a code editor with a dark theme. The editor has a tab labeled 'main.py' and buttons for 'Visualize' and 'Run'. The code in the editor is as follows:

```
1 def sumCounts(nums):  
2     total = 0  
3     for i in range(len(nums)):  
4         distinct = set()  
5         for j in range(i, len(nums)):  
6             distinct.add(nums[j])  
7             count = len(distinct)  
8             total += count * count  
9     return total  
10 nums = [1, 2, 1]  
11 print(sumCounts(nums))  
12 nums = [1, 1]  
13 print(sumCounts(nums))
```

The right side of the editor shows the 'Output' pane with the following text:

```
15  
3
```

4. def countPairs(nums, k):

```
    count = 0
```

```
    for i in range(len(nums)):
```

```
        for j in range(i + 1, len(nums)):
```

```
            if nums[i] == nums[j] and (i * j) % k == 0:
```

```
                count += 1
```

```
    return count
```

Example 1

```
nums = [3, 1, 2, 2, 2, 1, 3]
```

```
k = 2
```

```
print(countPairs(nums, k))
```

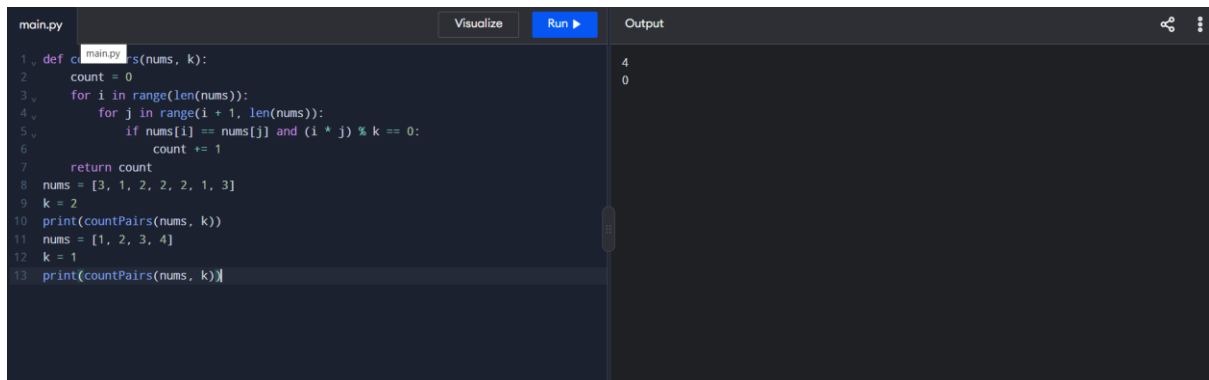
Example 2

```
nums = [1, 2, 3, 4]
```

```
k = 1
```

```
print(countPairs(nums, k))
```

OUTPUT:



The screenshot shows a code editor with a file named 'main.py'. The code defines a function `countPairs(nums, k)` that counts the number of pairs (i, j) such that `nums[i] == nums[j]` and `(i * j) % k == 0`. The function is tested with two sets of inputs: `nums = [3, 1, 2, 2, 2, 1, 3]` with `k = 2`, and `nums = [1, 2, 3, 4]` with `k = 1`. The output shows the results of these tests: 4 for the first case and 0 for the second case.

```
main.py Visualize Run ▶ Output
1 def countPairs(nums, k):
2     count = 0
3     for i in range(len(nums)):
4         for j in range(i + 1, len(nums)):
5             if nums[i] == nums[j] and (i * j) % k == 0:
6                 count += 1
7     return count
8 nums = [3, 1, 2, 2, 2, 1, 3]
9 k = 2
10 print(countPairs(nums, k))
11 nums = [1, 2, 3, 4]
12 k = 1
13 print(countPairs(nums, k))

4
0
```

5. def findMax(nums):

```
    maximum = nums[0]
```

```
    for i in range(1, len(nums)):
```

```
        if nums[i] > maximum:
```

```
            maximum = nums[i]
```

```
    return maximum
```

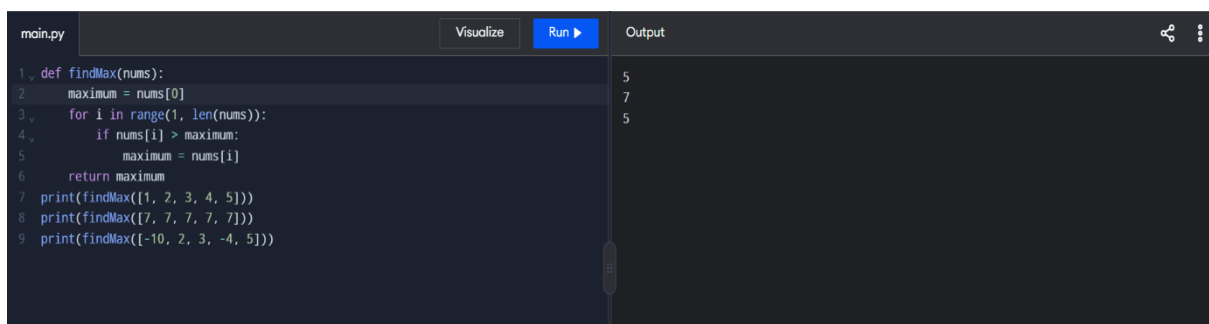
```
# Test cases
```

```
print(findMax([1, 2, 3, 4, 5]))
```

```
print(findMax([7, 7, 7, 7, 7]))
```

```
print(findMax([-10, 2, 3, -4, 5]))
```

OUTPUT:



The screenshot shows a code editor with a file named 'main.py'. The code defines a function `findMax(nums)` that returns the maximum value in the list `nums`. The function is tested with three sets of inputs: `[1, 2, 3, 4, 5]`, `[7, 7, 7, 7, 7]`, and `[-10, 2, 3, -4, 5]`. The output shows the results of these tests: 5, 7, and 5 respectively.

```
main.py Visualize Run ▶ Output
1 def findMax(nums):
2     maximum = nums[0]
3     for i in range(1, len(nums)):
4         if nums[i] > maximum:
5             maximum = nums[i]
6     return maximum
7 print(findMax([1, 2, 3, 4, 5]))
8 print(findMax([7, 7, 7, 7, 7]))
9 print(findMax([-10, 2, 3, -4, 5]))

5
7
5
```

6. def find_max(nums):

```
    if len(nums) == 0:
```

```
        return None
```

```
    nums.sort()      # Efficient sorting
```

```

    return nums[-1]    # Maximum element

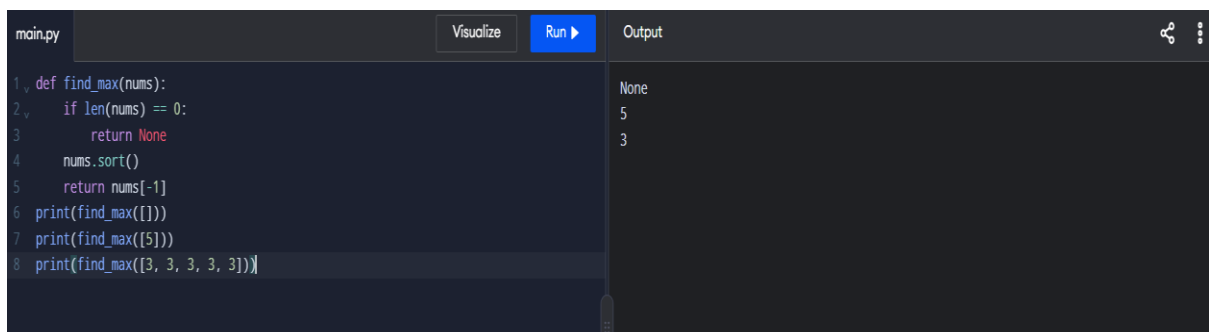
# Test Case 1
print(find_max([]))

# Test Case 2
print(find_max([5]))

# Test Case 3
print (find_max([3, 3, 3, 3, 3]))

```

OUTPUT:



```

main.py Visualize Run ▶ Output
1 def find_max(nums):
2     if len(nums) == 0:
3         return None
4     nums.sort()
5     return nums[-1]
6 print(find_max([]))
7 print(find_max([5]))
8 print(find_max([3, 3, 3, 3, 3]))

```

Output:

```

None
5
3

```

```

7. def unique_elements(nums):
    return list(set(nums))

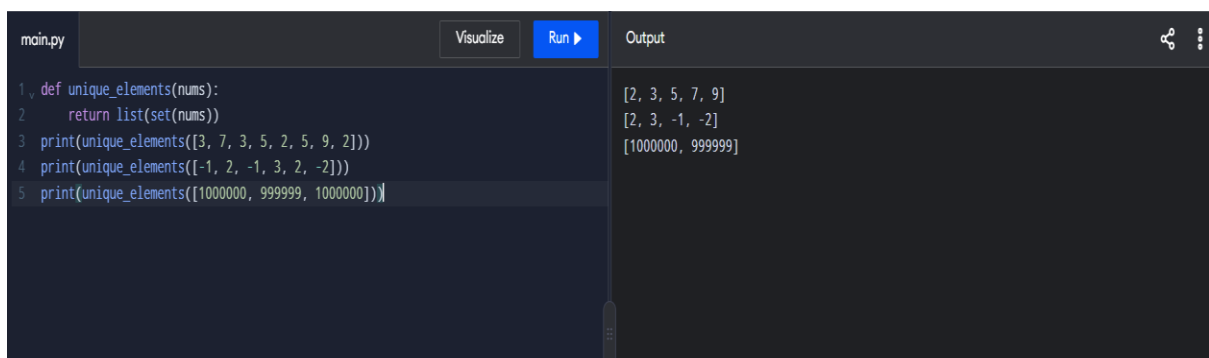
# Test Case 1
print(unique_elements([3, 7, 3, 5, 2, 5, 9, 2]))

# Test Case 2
print(unique_elements([-1, 2, -1, 3, 2, -2]))

# Test Case 3
print(unique_elements([1000000, 999999, 1000000]))

```

OUTPUT:



```

main.py Visualize Run ▶ Output
1 def unique_elements(nums):
2     return list(set(nums))
3 print(unique_elements([3, 7, 3, 5, 2, 5, 9, 2]))
4 print(unique_elements([-1, 2, -1, 3, 2, -2]))
5 print(unique_elements([1000000, 999999, 1000000]))

```

Output:

```

[2, 3, 5, 7, 9]
[2, 3, -1, -2]
[1000000, 999999]

```

```

8. def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                # Swap elements
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr

```

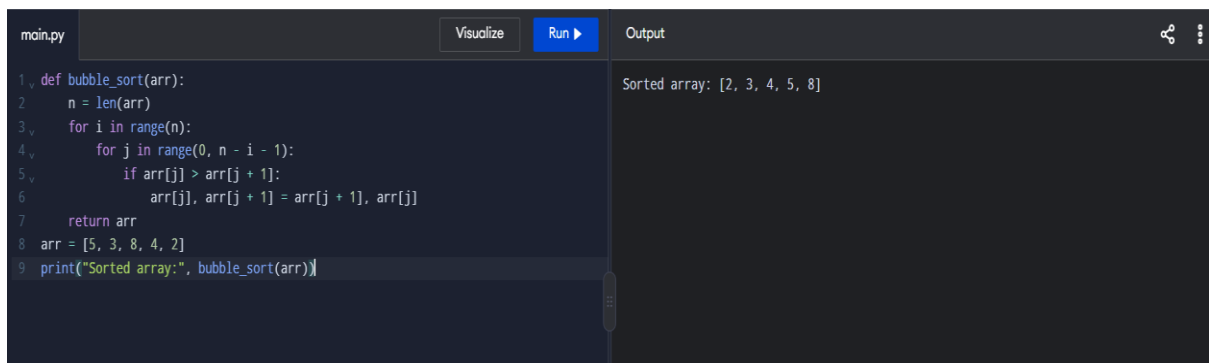
Example

```

arr = [5, 3, 8, 4, 2]
print("Sorted array:", bubble_sort(arr))

```

OUTPUT:



The screenshot shows a code editor with a dark theme. The left pane contains the Python code for the bubble_sort function and its execution. The right pane shows the output of the program. The code is as follows:

```

1 def bubble_sort(arr):
2     n = len(arr)
3     for i in range(n):
4         for j in range(0, n - i - 1):
5             if arr[j] > arr[j + 1]:
6                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
7     return arr
8 arr = [5, 3, 8, 4, 2]
9 print("Sorted array:", bubble_sort(arr))

```

The output pane displays the result: "Sorted array: [2, 3, 4, 5, 8]".

```

9. def binary_search(arr, key):

```

```

    low = 0

```

```

    high = len(arr) - 1

```

```

    while low <= high:

```

```

        mid = (low + high) // 2

```

```

        if arr[mid] == key:

```

```

            return mid

```

```

        elif arr[mid] < key:

```

```

            low = mid + 1

```

```

        else:

```

```

            high = mid - 1

```

```

    return -1

def search_element(arr, key):
    # Sort the array for binary search
    sorted_arr = sorted(arr)

    result = binary_search(sorted_arr, key)

    if result != -1:
        # Find original position for the required output
        original_position = arr.index(key) + 1

        print("Element", key, "is found at position", original_position)
    else:
        print("Element", key, "is not found")

# Test Case 1
X = [3, 4, 6, -9, 10, 8, 9, 30]
search_element(X, 10)

# Test Case 2
X = [3, 4, 6, -9, 10, 8, 9, 30]
search_element(X, 100)

```

OUTPUT:

The screenshot shows a code editor with a file named 'main.py'. The code implements a binary search function and tests it with two cases. The output window on the right shows the results of the function calls.

```

main.py Visualize Run
1 high = len(arr) - 1
2 while low <= high:
3     mid = (low + high) // 2
4     if arr[mid] == key:
5         return mid
6     elif arr[mid] < key:
7         low = mid + 1
8     else:
9         high = mid - 1
10 return -1
11
12 def search_element(arr, key):
13     sorted_arr = sorted(arr)
14     result = binary_search(sorted_arr, key)
15     if result != -1:
16         original_position = arr.index(key) + 1
17         print("Element", key, "is found at position", original_position)
18     else:
19         print("Element", key, "is not found")
20
21 X = [3, 4, 6, -9, 10, 8, 9, 30]
22 search_element(X, 10)
23 X = [3, 4, 6, -9, 10, 8, 9, 30]
24 search_element(X, 100)

```

Output

```

Element 10 is found at position 5
Element 100 is not found

```

10. def merge_sort(nums):

```

    if len(nums) <= 1:

```

```

        return nums

```

```

    mid = len(nums) // 2

```

```

left = merge_sort(nums[:mid])
right = merge_sort(nums[mid:])
result = []
i = 0
j = 0
while i < len(left) and j < len(right):
    if left[i] <= right[j]:
        result.append(left[i])
        i += 1
    else:
        result.append(right[j])
        j += 1
while i < len(left):
    result.append(left[i])
    i += 1
while j < len(right):
    result.append(right[j])
    j += 1
return result

# Example
nums = [5, 2, 8, 1, 3]
print(merge_sort(nums))

```

OUTPUT:

The screenshot shows a code editor with a dark theme. The file is named 'main.py'. The code defines a recursive merge_sort function. It includes a base case for lists of length 1 or less. The function splits the list into two halves, recursively sorts them, and then merges them back together by comparing elements from each half. The output of the function is displayed in the 'Output' panel on the right, showing the sorted list [1, 2, 3, 5, 8].

```

main.py Visualize Run ▶ Output
1 def merge_sort(nums):
2     if len(nums) <= 1:
3         return nums
4     mid = len(nums) // 2
5     left = merge_sort(nums[:mid])
6     right = merge_sort(nums[mid:])
7     result = []
8     i = 0
9     j = 0
10    while i < len(left) and j < len(right):
11        if left[i] <= right[j]:
12            result.append(left[i])
13            i += 1
14        else:
15            result.append(right[j])
16            j += 1
17    while i < len(left):
18        result.append(left[i])
19        i += 1
20    while j < len(right):
21        result.append(right[j])
22        j += 1

```

[1, 2, 3, 5, 8]